PSYC 51.17: Models of Language and Communication

Lecture 19: BERT Variants

Week 6, Lecture 2 - Improvements and Optimizations

PSYC 51.17: Models of Language and Communication

Winter 2026

Winter 2026

Today's Agenda

1. **RoBERTa**: Robustly Optimized BERT
2. **ALBERT**: A Lite BERT with parameter sharing
3. **DistilBERT**: Knowledge distillation for efficiency
4. **ELECTRA**: Replace Token Detection
5. **Comparative Analysis**: When to use which variant
6. **Practical Considerations**: Model selection guide

Goal: Understand improvements to BERT and choose the right model

Winter 2026

- 110M (Base) or 340M (Large) parameters
- Large memory footprint
- Slow inference

3. **Training Efficiency**

- Only 15% of tokens are predicted
- 85% of computation "wasted"?

2. **Dynamic Masking**

- Generate masking pattern every time
- BERT: static masks (same for every epoch)
- More diverse training signal

3. **Larger Batches, More Data**

- Batch size: 8K sequences (vs BERT's 256)

RoBERTa Results

Consistent improvements over BERT

SQuAD 2.0	**83.1**	**89.4**	**+6.3**
MNLI	86.7	90.2	+3.5
SST-2	94.9	96.4	+1.5
RACE	72.0	83.2	+11.2

Key Findings:

Dynamic vs Static Masking

How masking patterns are generated

Static Masking (BERT): Preprocessing:

- Mask tokens once
- Save masked dataset
- Same masks every epoch

Dynamic Masking (RoBERTa): On-the-fly:

- Generate masks during training
- Different masks each time
- More variety

ALBERT: A Lite BERT

Key idea: Parameter sharing for efficiency

ALBERT's Innovations (Lan et al. 2019):

1. Factorized Embedding

```
# BERT: Direct embedding
# 30K vocab × 768 hidden = 23M param
```

2. Cross-Layer Sharing

- All 12 layers share same weights
- 89% parameter reduction

ALBERT Parameter Efficiency

Dramatic parameter reduction!

ALBERT-base	**12**	**768**	**12M**
ALBERT-large	24	1024	18M
ALBERT-xlarge	24	2048	60M
ALBERT-xxlarge	12	4096	235M

Key Observations:

- ALBERT-base: than BERT-base
- Can train much larger hidden sizes with same memory
- ALBERT-xxlarge: 4096 hidden dim, still only 235M params
- Trade-off: fewer params but similar computation (layer sharing)

Cross-Layer Parameter Sharing

How ALBERT achieves parameter efficiency

BERT (No Sharing):

```
class BERT:
 def __init__(self):
  # Each layer has unique parameters
  self.layers = [
```

ALBERT (Full Sharing):

```
class ALBERT:
 def __init__(sel
  # Single shared layer!
  self.shared_laye
```

DistilBERT: Knowledge Distillation

Key idea: Train small model to mimic large model

```
# Knowledge Distillation Training Loop
teacher = BertModel.from_pretrained("bert-base") # 12 layers, frozen
student = DistilBertModel(num_layers=6) # 6 layers, trainable

for batch in training_data:
 # Teacher provides "soft targets" (probability distributions)
 with torch.no_grad():
 teacher_logits = teacher(batch) # e.g., [0.7, 0.2, 0.1, ...]

 # Student tries to match teacher's distribution
 student_logits = student(batch)

 # Distillation loss: KL divergence between distributions
 # Temperature T=2 softens the distribution (more informative)
 loss_distill = KL_divergence(
 softmax(student_logits / T),
```


continued...

DistilBERT: Knowledge Distillation

```
    softmax(teacher_logits / T)
    )

    # Also include MLM loss for language modeling
    loss_mlm = masked_lm_loss(student_logits, labels)

    # Combined loss
    loss = 0.5 * loss_distill + 0.5 * loss_mlm
```

Why soft targets work: Teacher's "wrong" predictions contain information (e.g., "dog" → "cat" more likely than "car")

Reference: Sanh et al. (2019) - "DistilBERT, a distilled version of BERT"

...continued

DistilBERT Results

Significant efficiency gains!

Inference Speed	**1x**	**1.6x**	**60% faster**
GLUE Score	79.6	77.0	97% retained

Performance on Specific Tasks:

SST-2 (Acc)	**94.9**	**92.7**

ELECTRA: Efficient Learning

Key idea: Learn from all tokens, not just 15%

```
# ELECTRA Training: Generator + Discriminator setup
```

Efficiency Gain

BERT: Learns from 15% of tokens (masked ones only)
ELECTRA: Learns from 100% of tokens (all get labeled)

Result: Same quality with 4x less compute!

Reference: Clark et al. (2020) - "ELECTRA: Pre-training Text Encoders as Discriminators"

ELECTRA Benefits

More efficient pre-training

Advantages:

1. **Sample Efficiency**

- Learn from all tokens (100%) vs only masked (15%)
- Reaches same performance with less data
- Faster convergence

2. **Better Performance**

BERT Variants Comparison

Summary of key variants

p{3cm}p{3cm}} Model	Key Innovation	Advantages	Best For

General Guidelines:

- **Best quality:** RoBERTa-Large
- **Best efficiency:** DistilBERT
- **Limited memory:** ALBERT

2. **ERNIE (Baidu, 2019)**

- Entity-level and phrase-level masking
- Knowledge enhancement
- Strong on Chinese NLP tasks

3. **SpanBERT (Facebook, 2019)**

- Mask random spans instead of random tokens
- Span boundary objective

Model Selection Guide

How to choose the right model for your task

```
Start → Quality or Speed? → RoBERTa-Large → Memory? → DistilBERT →
```

Additional Considerations:

- **Domain:** Consider domain-specific pre-trained models (BioBERT, SciBERT, etc.)
- **Language:** Multilingual? Use mBERT, XLM-R
- **Task type:** Generation? Consider BART/T5 instead

Using Different BERT Variants

Easy switching with HuggingFace

```python
from transformers import AutoModel, AutoTokenizer

# BERT
bert_model = AutoModel.from_pretrained("bert-base-uncased")
bert_tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# RoBERTa
roberta_model = AutoModel.from_pretrained("roberta-base")
roberta_tokenizer = AutoTokenizer.from_pretrained("roberta-base")

# ALBERT
albert_model = AutoModel.from_pretrained("albert-base-v2")
albert_tokenizer = AutoTokenizer.from_pretrained("albert-base-v2")

# DistilBERT
```


continued...

Using Different BERT Variants

```
distilbert_model = AutoModel.from_pretrained("distilbert-base-uncased")
distilbert_tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

# ELECTRA
electra_model = AutoModel.from_pretrained("google/electra-base-discriminator")
electra_tokenizer = AutoTokenizer.from_pretrained("google/electra-base-discriminator")

# All have the same API!
inputs = tokenizer("Hello, my dog is cute", return_tensors="pt")
outputs = model(**inputs)
```


...continued

Benchmarking BERT Variants: Worked Example

Practical comparison on sentiment analysis

```
import time
from transformers import pipeline

# Load different models for sentiment analysis
models = {
 "bert-base": "textattack/bert-base-uncased-SST-2",
 "distilbert": "distilbert-base-uncased-finetuned-sst-2-english",
 "albert": "textattack/albert-base-v2-SST-2",
}

test_texts = ["This movie was fantastic!", "I hated every minute of it."] * 100

for name, model_id in models.items():
 pipe = pipeline("sentiment-analysis", model=model_id)

 start = time.time()
 results = pipe(test_texts)
 elapsed = time.time() - start
```


continued...

Benchmarking BERT Variants: Worked Example

```
19
20  print(f"{name}: {elapsed:.2f}s for 200 samples ({200/elapsed:.1f} samples/sec)")
```

Typical Results:

Model	Accuracy	Speed (samples/sec)	Memory
bert-base	93.2%	45	420MB
distilbert	91.3%	85	250MB

...continued

- ALBERT shares all layers. Why does this work?
- What are the limits of parameter sharing?
- Is there a "sweet spot"?

3. **Knowledge Distillation:**

- Why does student learn better from teacher than from labels?

Looking Ahead

Today we learned:

- RoBERTa: Better training matters
- ALBERT: Parameter sharing for efficiency
- DistilBERT: Knowledge distillation
- ELECTRA: Replace token detection
- How to choose the right variant

Next lecture (Lecture 17 - Applications of Encoder Models):

**From theory to practice! **

Winter 2026

- Parameter sharing dramatically reduces model size
- Factorized embeddings for efficiency
- 89% fewer parameters than BERT

3. **DistilBERT**

- Knowledge distillation for deployment
- 40% smaller, 60% faster, 97% performance

- **Lan et al. (2019)** - "ALBERT: A Lite BERT for Self-supervised Learning"
- **Sanh et al. (2019)** - "DistilBERT, a distilled version of BERT"
- **Clark et al. (2020)** - "ELECTRA: Pre-training Text Encoders as Discriminators"
- **He et al. (2020)** - "DeBERTa: Decoding-enhanced BERT with Disentangled Attention"

Resources:

- HuggingFace Model Hub:

Questions?

Discussion Time

Topics for discussion:

- BERT variants and improvements
- Model selection strategies
- Knowledge distillation
- Parameter efficiency
- Implementation questions

Thank you!

Next: Applications and Real-World Use Cases!